Lecture Notes: The Execution Model, Stack and Heap

High Performance Computing — Università di Trieste, 2025/2026

Motivation

Before we can meaningfully optimise code, we need to understand *where* our data live and *how* the operating system organises memory for a running process. This may seem like a detour from performance tuning, but it is not: the distinction between stack and heap has direct consequences on data locality, allocation cost, and ultimately on the efficiency of your programs — all things that matter enormously when you target HPC systems.

In this lecture we focus on Unix/Linux systems, which are the de facto standard on every HPC facility you will ever encounter, from your department's small cluster to a tier-0 machine like CINECA's Leonardo. The big picture, however, remains valid for other operating systems as well.

Part 1 — The Execution Model on Linux

What happens when you run a program

When you type `./my_program` on a terminal, a rather complex chain of events takes place. Let us sketch the essential steps.

The shell forks a child process and calls the `execve()` system call (or one of its wrappers). The kernel then:

1. Reads the ELF binary (the compiled executable) from disk. The ELF (Executable and Linkable Format) is the standard binary format on Linux; it contains the machine code, the data, the symbol table, and all the metadata the kernel needs to set up the process.
2. Creates a new *process*, which is essentially a data structure (`task_struct` in the Linux kernel) that holds all the bookkeeping information: process ID, credentials, file descriptors, signal handlers, and — crucially for us — the description of the process's address space.
3. Sets up the **virtual address space** for this new process.
4. Loads the executable code (the `.text` segment) into memory, together with any shared libraries the program depends on (the dynamic linker `ld-linux.so` resolves these).
5. Initialises the data segments, the stack, and hands control to the program's entry point (which is actually `_start` from the C runtime library, which sets up the environment and eventually calls `main()`).

At this point the process is running. The CPU fetches instructions from the text segment, reads and writes data in the various memory regions, and the kernel mediates access to all other resources (I/O, network, devices, etc.).

The key observation is that the compiled binary alone is not sufficient to run. The program needs *memory* for code and data, access to *CPU time*, and a way to interact with the external world. All of this is provided by the operating system.

The virtual address space: your "sandbox"

Every process on a modern OS runs inside its own **virtual address space**. This is a fundamental abstraction: from the point of view of your program, it sees a large, contiguous range of memory addresses that starts at zero and extends up to some maximum. The program has no direct knowledge of physical memory — it only ever manipulates virtual addresses.

This design serves several purposes. It isolates processes from each other (process A cannot accidentally — or maliciously — corrupt the memory of process B). It simplifies programming, because every program sees the same,

clean address space regardless of how fragmented or complex the physical memory might be. And it enables important features like demand paging, memory-mapped files, and copy-on-write.

The *size* of the virtual address space depends on the architecture and, more precisely, on how many bits of the virtual address are actually implemented in hardware:

Architecture	Paging levels	VA bits used	Virtual space
32-bit x86	2	32	4 GiB
64-bit x86-64, 4-level paging	4	48	256 TiB
64-bit x86-64, 5-level paging (LA57)	5	57	128 PiB
64-bit, full (theoretical)	—	64	16 EiB

A clarification is in order here. The original x86-64 architecture uses 4-level paging, which provides 48-bit virtual addresses (256 TiB). Starting with Intel Ice Lake (2019), the LA57 extension introduces 5-level paging with 57-bit virtual addresses (128 PiB). Note that 57 — not 56 — is the correct number of bits: five levels of 9-bit indices plus a 12-bit page offset give 9×5 + 12 = 57. The full 64-bit range (16 EiB) has never been implemented and likely never will be in the current page-table scheme.

On current HPC systems, 48-bit addressing (256 TiB) is the common case. The 5-level paging extension exists primarily because some vendors already sell servers with physical memory approaching 64 TiB, which is the physical memory limit of 4-level paging.

The point, however, is that this is the *addressing capability*, not the actual physical memory available. Your process might have a 256 TiB virtual space in principle, but obviously only a fraction of it will be backed by real physical RAM at any given instant. As the Linux kernel documentation puts it, with reference to the 128 PiB limit of 5-level paging: this "ought to be enough for anybody."

From virtual to physical: paging and the TLB

The virtual addresses that your program uses must be translated into physical addresses before the memory controller can actually access DRAM. This translation is performed via the **paging mechanism**.

The idea is conceptually straightforward: both virtual and physical memory are divided into fixed-size blocks called **pages**. On x86-64, the base page size is 4 KiB (4096 bytes). On HPC systems with large memory, **huge pages** of 2 MiB or 1 GiB are commonly used to reduce translation overhead (we will see why shortly). Each virtual page is mapped to a physical page (also called a "frame"), and the mapping is stored in a hierarchical structure called the **page table**, maintained by the kernel.

`<non-required details>` On x86-64 with 4-level paging, the virtual address is split into five fields: four 9-bit indices (one for each level of the page table hierarchy: PML4, PDPT, PD, PT) plus a 12-bit offset within the page. Each level of the hierarchy contains 512 entries ($2^9 = 512$), and each entry — a Page Table Entry (PTE) — records the physical address of the next level (or of the final page frame) plus metadata (present bit, read/write permissions, dirty bit, accessed bit, etc.). Walking this four-level hierarchy is called a **page table walk**, and it requires up to four sequential memory accesses. `</non-required details>`

A single physical page can be mapped into the virtual address space of *multiple* processes. This is how shared libraries work: `libc.so` is loaded once in physical memory but appears in the virtual space of every process that uses it. This mechanism is also fundamental for shared memory segments in parallel computing (think of `mmap()` or POSIX shared memory, which you will encounter when we discuss inter-process communication and MPI shared-memory windows).

Now, walking the four-level page table on every memory access would be intolerably slow. To avoid this overhead, the CPU contains a specialised cache for page table entries called the **Translation Lookaside Buffer (TLB)**.

`<non-required details>` On modern processors, the TLB is itself a multi-level structure. Let us take Intel Skylake (the microarchitecture of many current Xeon processors in HPC) as a concrete example:

L1 DTLB (data): 64 entries for 4 KiB pages (4-way associative), 32 entries for 2 MiB pages (4-way), 4 entries for 1 GiB pages (4-way). Hit latency: folded into the L1 cache access (~4–5 cycles total).

L1 ITLB (instructions): 128 entries for 4 KiB pages (8-way), 8 entries for 2 MiB pages (fully associative).

L2 STLB (unified, shared between data and instructions): 1536 entries (12-way associative), supports 4 KiB and 2 MiB pages. Miss penalty from L1 TLB to L2 STLB: approximately 9 cycles (measured on Skylake). `</non-required details>`

If the translation is not found in any TLB level, a full page table walk is triggered. This walk is performed by dedicated hardware (the Memory Management Unit, MMU), and its cost depends on where the page table entries are cached. In the best case (PTE in L1 data cache), the walk can complete in ~10–15 cycles. In the worst case (PTE in main memory), it can cost several hundred cycles — comparable to a full DRAM access.

TLB miss rates are usually low — around 1–2% for well-behaved workloads. But when they occur, the cost is severe. This has direct practical consequences for HPC programmers: if your data access pattern touches many different pages in a scattered fashion, you will generate TLB misses. Using huge pages (2 MiB instead of 4 KiB) dramatically helps, because each TLB entry covers 512× more memory. On many HPC systems, huge pages are configured through `libhugetlbfs` or transparent huge pages (THP) for this very reason.

Part 2 — The Layout of a Process in Memory

Let us now look at what the virtual address space of a typical Linux process actually contains. We present the layout for a modern x86-64 system with 48-bit virtual addressing (4-level paging), which is the standard on current HPC hardware. On such a system, the 64-bit virtual address space is split into two halves by the **canonical address** rule: bits 63 through 48 must all be copies of bit 47 (sign extension). This creates two valid regions — the *low canonical addresses* (bits 63:48 all zero, used for user space) and the *high canonical addresses* (bits 63:48 all one, used for the kernel) — separated by a gigantic non-canonical hole in the middle that no one can access.

The user-space portion spans from `0x0000'0000'0000'0000` to `0x0000'7FFF'FFFF'FFFF` — a total of 128 TiB. The kernel occupies the high end, from `0xFFFF'8000'0000'0000` to `0xFFFF'FFFF'FFFF'FFFF`. User code cannot access the kernel range; attempting to do so causes a page fault.

Within the user-space 128 TiB, the process memory is organised as follows:

```
 0x0000'7FFF'FFFF'FFFF   ← top of user space (128 TiB)
┌──────────────────────────┐
│                          │
│   Stack                  │  ← grows downward (toward lower addresses)
│          ↓               │    start address randomised by ASLR
│                          │
│   (unmapped guard pages) │
│                          │
│  Memory-mapped region    │  ← shared libraries, mmap'd files, large mallocs
│                          │    grows downward (toward the heap)
│                          │
│   ~~~~~~~~~~~~~~~~~~~~~~~~  (enormous unmapped gap, tens of TiB)
```

```
│                          │
│          ↑               │
│   Heap (brk/sbrk)        │  ← grows upward (toward higher addresses)
│                          │
├──────────────────────────┤    random brk offset (ASLR)
│   BSS  (uninitialised    │
│         global data)     │
├──────────────────────────┤
│   Data (initialised      │
│         global data)     │
├──────────────────────────┤
│   Text (executable code) │
├──────────────────────────┤    ~ 0x0000'0040'0000 (non-PIE default)
│   (reserved / NULL page) │    or randomised base (PIE, default on modern distros)
└──────────────────────────┘
 0x0000'0000'0000'0000   ← address zero (unmapped, NULL dereference trap)
```

A few remarks on the addresses. On 64-bit Linux, the default load address for a non-position-independent executable (non-PIE) is `0x400000`. However, modern distributions compile executables as PIE (Position-Independent Executables) by default, which means the text segment base is randomised at each execution by ASLR. Similarly, the stack start, heap start (the initial `brk`), and the mmap region base are all randomised. You can disable ASLR for debugging with `setarch $(uname -m) -R ./my_program`, but in production the randomisation is always active.

Let us walk through each region from bottom to top.

Reserved region at address 0. The very first page (or pages) at the bottom of the address space is intentionally left unmapped. This is so that dereferencing a NULL pointer (which is defined as address 0 in C/C++) causes an immediate segmentation fault, rather than silently reading whatever happens to be at that address. On Linux, the minimum mappable address is controlled by `vm.mmap_min_addr` (typically 65536, i.e. the first 16 pages are unmapped).

Text segment (.text). This contains the actual machine code of your program — the compiled instructions. When you write `func1()`, `func2()`, `main()`, they all end up here, translated into the binary opcodes that the CPU will execute. This segment is typically marked as read-only and executable. It can be shared among all instances of the same program running concurrently.

Data segment (.data, initialised). Global and static variables that you initialise at declaration time live here. For instance:

```
double pi = 3.14159265358979323846;
static int run_count = 1;
```

The values are known at compile time and are stored directly in the executable binary; when the process starts, they are loaded into this segment.

BSS segment (.bss, uninitialised). Global and static variables that are declared but *not* explicitly initialised go here. For instance:

```
double result_of_calculation;
int iteration_counter;
static int days_of_the_week;
```

The compiler knows their types and sizes but not their initial values. The BSS segment is zeroed out by the OS at process startup. The name BSS is a historical relic: it stands for "Block Started by Symbol," a pseudo-operation from the UA-SAP assembler for the IBM 704, dating back to the mid-1950s. Peter van der Linden, in his book *Expert C Programming*, suggests the mnemonic "Better Save Space" — because the BSS segment does not actually occupy space in the object file on disk (only its total size is recorded), unlike the .data segment where every initialised byte must be stored.

Heap. This is the region used for dynamic memory allocation — the memory you get from `malloc()`, `calloc()`, `new`, etc. Traditionally, the heap grows *upward* (toward higher addresses) through the `brk()` / `sbrk()` system calls. In modern glibc, large allocations are often served via `mmap()` instead, placing them in the memory-mapped region. We will discuss the heap in detail below.

Memory-mapped region. Between the heap and the stack lies a large region used for memory-mapped files and anonymous mappings. This is where shared libraries are loaded (each `.so` file is `mmap()`'d into this region), and where large `malloc()` allocations end up (glibc's `malloc` uses `mmap()` for allocations above a certain threshold, typically 128 KiB). This region is also used for POSIX shared memory (`shm_open()` + `mmap()`), which is relevant for parallel computing.

Stack. This is the region used for function call management — local variables, return addresses, function arguments. The stack grows *downward* (toward lower addresses). This is the other main character of our story.

The gap between the heap/mmap region and the stack is unmapped: accessing it will trigger a segmentation fault. With 128 TiB of user-space address range available, the gap between the top of the heap and the bottom of the mmap/stack region is typically tens of terabytes wide — heap-stack collisions are not a realistic concern on 64-bit systems under any normal workload.

Part 3 — The Stack

What the stack is for

The stack is the region of memory dedicated to **local, function-scoped** data. Every time a function is called, a new **stack frame** is pushed onto the stack containing:

1. The function's arguments (with caveats — see below).
2. The **return address**, i.e. the address of the instruction to resume execution at when the function returns.
3. The saved frame pointer of the caller (when frame pointers are used).
4. All the **local variables** declared inside the function.

The CPU manages the stack via dedicated registers. On x86-64 these are:

RSP (Stack Pointer): always points to the "top" of the stack — i.e. the lowest address currently in use. Every `push` instruction decrements RSP by 8 and writes a 64-bit value; every `pop` increments RSP by 8 and reads.

RBP (Base Pointer, also called Frame Pointer): points to a fixed reference point within the current stack frame. Local variables and function arguments are accessed as fixed offsets from RBP.

The reason for having *two* registers is convenience: RSP moves as you push and pop temporaries, but RBP stays fixed for the duration of the function, providing a stable anchor for addressing local variables. It also enables debuggers and profilers to walk the call stack by following the chain of saved RBP values.

However — and this is important for understanding compiler output — on x86-64, GCC omits the frame pointer by default when optimisation is enabled (`-O1` and above). This is because the AMD64 ABI explicitly states that the use of RBP as a frame pointer is optional. When the frame pointer is omitted, the compiler uses RSP-relative addressing for all local variables, and RBP becomes available as a general-purpose register (giving the compiler one extra

register to work with, which can matter in tight loops). If you want frame pointers for debugging, use `-fno-omit-frame-pointer`. This flag is, incidentally, recommended for profiling with tools like `perf` — without it, stack traces become unreliable.

A concrete example: function calls and the stack

Consider the following call chain:

```
void func2(int x) {
    int local_b = x * 2;
    // ... do something ...
}

void func1(int n) {
    double a = 3.14;
    int j = 42;
    func2(j);
    // ... continue ...
}

int main() {
    func1(10);
    return 0;
}
```

When `main()` calls `func1(10)`, the integer 10 is passed in the register `RDI` (not on the stack — more on this below). The `call` instruction pushes the return address onto the stack and jumps to `func1`. The function prologue of `func1` (if frame pointers are enabled) saves the old RBP, sets RBP to the current RSP, and decrements RSP to make room for local variables. At this point:

```
 Higher addresses
┌──────────────────────────┐
│  main()'s stack frame    │
├──────────────────────────┤
│  return address → main   │  (pushed by the call instruction)
│  saved RBP of main       │  (pushed by func1's prologue)
│  double a = 3.14         │  ← RBP points near here
│  int j = 42              │
│  (padding for alignment) │  ← RSP points here
└──────────────────────────┘
 Lower addresses
```

When `func1()` then calls `func2(j)`, the value of `j` (42) is loaded into `RDI`, and the stack grows further:

```
 Higher addresses
┌──────────────────────────┐
│  main()'s stack frame    │
├──────────────────────────┤
│  func1()'s stack frame   │
│    (a, j, etc.)          │
├──────────────────────────┤
```

```
│  return address → func1  │
│  saved RBP of func1      │
│  int local_b             │  ← RSP points here
└──────────────────────────┘

 Lower addresses
```

When `func2()` returns, its frame is simply discarded: RSP and RBP are restored to their values before the call. The memory that was occupied by `func2`'s local variables is still physically there, but it is now *below* RSP and therefore logically invalid — it can (and will) be overwritten by the next function call.

Argument passing on x86-64 (System V AMD64 ABI). On the x86-64 architecture under Linux and macOS, the calling convention is defined by the System V AMD64 ABI. The first six integer or pointer arguments are passed in registers, in this order: `RDI`, `RSI`, `RDX`, `RCX`, `R8`, `R9`. Floating-point arguments use `XMM0` through `XMM7` (up to 8 floating-point arguments in registers). Only the 7th integer argument and beyond, or structures that do not fit in registers, spill to the stack. This is an important optimisation — it avoids memory accesses for the most common case.

Note that the Windows x64 calling convention is different: it uses only four registers (`RCX`, `RDX`, `R8`, `R9`) for the first four integer arguments, and has no red zone (see below). *This is one of many reasons why code compiled for Linux and Windows is not binary-compatible even on the same hardware.*

How variables are addressed on the stack

Since every local variable sits at a known, fixed offset from either RBP or RSP, the compiler can generate very efficient code to access them. Consider:

```c
void function(void) {
    double a;
    int j;
    int k;
    // ...
}
```

The compiler knows, at compile time, the exact layout. Suppose `a` is 8 bytes, `j` and `k` are 4 bytes each. Then, with frame pointer enabled, the layout relative to RBP is:

```
RBP → ┌────────────────────┐
      │  saved old RBP     │  [RBP + 0]
      ├────────────────────┤
      │  double a (8B)     │  [RBP - 8]
      ├────────────────────┤
      │  int j (4B)        │  [RBP - 12]
      ├────────────────────┤
      │  int k (4B)        │  [RBP - 16]
RSP → └────────────────────┘
```

Accessing any of these is a single instruction with a register-relative addressing mode:

```
MOVSD xmm0, QWORD PTR -8[rbp]     ; load a
MOV   eax,  DWORD PTR -12[rbp]    ; load j
MOV   ecx,  DWORD PTR -16[rbp]    ; load k
```

This is extremely fast — it is just an offset from a register that the CPU already holds. The offsets are known at compile time and encoded directly in the instruction.

The scope of the stack

The stack's scope is *strictly hierarchical*. A function can only access:

- Its own local variables (obviously).
- The stack of its *callers*, **if** a pointer to some caller's data has been explicitly passed as an argument. For instance, if `func2()` receives a pointer to a local variable of `func1()`, it can dereference it — the memory is valid because `func1()`'s frame is still alive on the stack above `func2()`'s frame.
- It **cannot** access the stack of sibling functions or of functions that have already returned.

This point deserves emphasis. Consider:

```
void function_1() { int x = 42; /* ... */ }
void function_2() { /* can function_2 access x? NO. */ }

int main() {
    function_1();   // x exists only while function_1 runs
    function_2();   // function_1's stack frame is gone; its memory
                    // has likely been overwritten by function_2's frame
    return 0;
}
```

When `function_1()` returns, its stack frame is deallocated. The memory it occupied is reused by `function_2()`. If `function_1()` had returned a pointer to its local variable `x`, that pointer would be **dangling** — it would point to memory that no longer belongs to `function_1()` and will be overwritten unpredictably. The compiler will warn you:

```
warning: function returns address of local variable
```

This is not just an academic concern. I have seen production codes in astrophysics that passed around pointers to stack-allocated buffers, and they "worked" for years because by coincidence the memory was not immediately overwritten.

As for other stack misuses (see next paragraph), that is an insidious class of bugs. One day, a new compiler version or optimisation level changed the stack layout, and the code started producing wrong results. Silently. No crash, no warning at runtime — just incorrect numbers in the simulation output.

Why the stack is purposely small

On a typical Linux system, the default stack size is 8 MiB. You can check it:

```
$ ulimit -s
8192          # value is in KiB, so 8192 KiB = 8 MiB
```

This may seem absurdly small compared to the gigabytes or terabytes of RAM available on an HPC node. But the stack is *intentionally* kept small, and there are good reasons for this.

First, the stack is meant for lightweight, transient data: loop counters, a few pointers, small local buffers. Its raison d'être is to be fast, and it achieves this partly by being small enough that the active portion fits comfortably in the L1 or L2 cache. If you pile gigabytes of data onto the stack, you defeat its purpose.

Second, in a multi-threaded program (and HPC is all about threads), **each thread gets its own stack**. If you run 128 OpenMP threads and each has a 1 GiB stack, you have just consumed 128 GiB of memory for stacks alone, before storing a single byte of useful data. The default stack size is a reasonable trade-off. (The default per-thread stack size in pthreads/OpenMP is typically 2–8 MiB, controlled by `OMP_STACKSIZE` for OpenMP threads.)

Third, a small stack acts as a safety net. Stack overflow (where the stack grows beyond its limit) typically causes an immediate segmentation fault, which is loud and obvious. The kernel places a guard page (an unmapped page) just beyond the stack limit, so any attempt to write past it triggers a page fault that the kernel translates into a `SIGSEGV`. If the stack could grow without bounds, a runaway recursion or an accidental large allocation would silently eat all available memory before the system noticed.

You *can* change the soft limit from the shell:

```
$ ulimit -s 16384    # set to 16 MiB
```

There is also a hard limit that the system administrator can set:

```
$ ulimit -Hs         # show hard limit (often "unlimited")
```

A technical aside: why is "unlimited" often represented as the value −1 (or, more precisely, `RLIM_INFINITY`, which is `(rlim_t)(-1)`)? Because `rlim_t` is an unsigned integer type, casting −1 to it gives the maximum representable value (all bits set to 1), which is the natural sentinel for "no limit." This is a common pattern in systems programming.

You can also query and modify the stack limit programmatically:

```
#include <sys/resource.h>

struct rlimit limits;
getrlimit(RLIMIT_STACK, &limits);
printf("Current: %lu, Max: %lu\n",
       (unsigned long)limits.rlim_cur,
       (unsigned long)limits.rlim_max);

// Enlarge the stack to 16 MiB (if rlim_max allows it)
limits.rlim_cur = 16 * 1024 * 1024;
setrlimit(RLIMIT_STACK, &limits);
```

But if you find yourself needing to enlarge the stack, you should stop and ask: *why?* In almost all cases, the correct answer is to move the large data to the heap using `malloc()` / `calloc()`. Needing a large stack is a design smell.

The perils of legacy code: stack abuse

In legacy Fortran codes (and in some poorly written C codes), it was common practice to declare large arrays as local variables:

```
void compute_something(int n) {
    double matrix[1000][1000];   // 8 million bytes on the stack!
    // ...
}
```

This single array is 1000 × 1000 × 8 = 8,000,000 bytes ≈ 7.6 MiB, which almost fills the entire default stack by itself. If there are *any* other local variables or if this function is not the only frame on the stack, the program will crash.

In Fortran 77/90 codes that predate dynamic allocation (i.e., before Fortran 90's `ALLOCATE` and Fortran 2003's improved pointer semantics), large local arrays are ubiquitous. Even modern Fortran compilers can place local arrays on the stack by default, unless you explicitly use `-heap-arrays` (Intel Fortran) or similar flags. This is a recurring source of mysterious segfaults when porting legacy physics codes to new systems.

Even smaller allocations can be problematic if the function is called recursively or from a deep call chain. Each level of recursion adds its own frame, and the cumulative effect can exhaust the stack.

Dynamic allocation on the stack: `alloca()` and VLAs

The function `alloca()` provides dynamic allocation *on the stack*:

```
#include <alloca.h>   // or just rely on the compiler built-in

void some_function(int n) {
    // n is supposed to be small
    double *temp = (double *)alloca(n * sizeof(double));
    // temp lives on the stack, is freed automatically on return
    // do NOT call free() on it — it would be undefined behaviour
}
```

This works by simply decrementing RSP by the requested amount. The memory is automatically reclaimed when the function returns (RSP is restored). The advantage over `malloc()` is zero allocation overhead — no allocator bookkeeping, no system calls, no potential contention in a multi-threaded context.

However, `alloca()` comes with significant risks. If `n` is unexpectedly large, you get a stack overflow with no graceful recovery. Unlike `malloc()`, there is no return value to check — `alloca()` does not fail cleanly; it just corrupts the stack or causes a segfault. Also, `alloca()` is **not part of the C standard** — it is a POSIX/glibc extension (and a GCC built-in). Portability is not guaranteed.

C99 introduced **Variable-Length Arrays (VLAs)**, which provide similar functionality within the language standard:

```
void some_function(int n) {
    double temp[n];   // C99 VLA, allocated on the stack
    // ...
}                     // automatically deallocated
```

However, VLAs were made *optional* in C11, and many coding standards (including CERT C and the Linux kernel coding style) discourage or outright forbid them for the same safety reasons as `alloca()`. The rule of thumb: if you are not absolutely sure that the allocation is small, use the heap.

Undefined behaviour: accessing below RSP

Everything below the current value of RSP is, in principle, *no man's land*. The memory is physically present (at least until the guard page), but it is **not** part of any valid stack frame. Accessing it is undefined behaviour.

`<non-required details>` There is, however, a subtlety specific to x86-64 on Linux and macOS. The System V AMD64 ABI defines a so-called **red zone**: a 128-byte area immediately below RSP that is guaranteed not to be modified by signal handlers or interrupt handlers (in user space). Leaf functions — functions that do not call any other

functions — can use this area as scratch space without adjusting RSP. This saves two instructions (the `sub rsp` in the prologue and the `add rsp` in the epilogue), which matters in small, hot functions.

The red zone is an ABI convention, not a hardware feature. Signal handlers respect it because the kernel subtracts 128 bytes from RSP before setting up the signal frame. But interrupts in **kernel mode** do not respect it — the CPU pushes the interrupt frame directly at RSP. This is why the Linux kernel is compiled with `-mno-red-zone`: without this flag, leaf functions in the kernel would have their below-RSP scratch space silently corrupted by hardware interrupts. This caused real bugs in early x86-64 kernel development, where random data corruption was traced to interrupt handlers overwriting the red zone.

The Windows x64 ABI does *not* have a red zone at all. Memory below RSP is explicitly volatile and may be overwritte at any time. `</non-required details>`

For user-space HPC code, the practical takeaway is: never write C code that deliberately accesses memory below the stack pointer beyond what the compiler generates. The red zone is managed by the compiler, not by you.

What happens if you access memory below RSP outside the red zone? The most benign outcome is that you read stale data from a previous function call. A more likely outcome, in a multi-threaded context, is that a signal handler fires, overwrites the memory you thought was safe, and you get silently corrupted data. There is no defined behaviour here — anything can happen, and none of it is good.

Stack smashing

Stack smashing (or stack buffer overflow) is one of the most classic and well-studied vulnerabilities in computing. The idea is simple: if a function writes beyond the bounds of a local buffer, it can overwrite adjacent data on the stack — including the saved return address.

Consider:

```
void vulnerable_function(char *input) {
    char buffer[64];
    strcpy(buffer, input);   // no bounds checking!
}
```

If `input` is longer than 64 bytes, `strcpy()` will happily keep writing past the end of `buffer`, overwriting whatever comes next on the stack: padding, other local variables, the saved frame pointer, the return address, and so on. An attacker who controls the content of `input` can craft it so that the overwritten return address points to malicious code (injected in the buffer itself, or elsewhere in memory).

This is the basis of the classic **buffer overflow attack**, described in the famous 1996 article "Smashing the Stack for Fun and Profit" by Aleph One (Phrack Magazine, issue #49, article 14). The article is worth reading as historical background and as a masterpiece of technical writing — it remains relevant nearly three decades later.

Modern systems deploy multiple countermeasures:

Stack canaries (stack protectors). The compiler inserts a random value (the "canary") between the local variables and the saved return address during the function prologue. Before the function returns, it checks whether the canary has been modified. If it has, the program aborts immediately with:

```
*** stack smashing detected ***: terminated
```

GCC enables this with `-fstack-protector` (protects functions with local arrays), `-fstack-protector-strong` (broader coverage, default in most distributions), or `-fstack-protector-all` (every function, at some performance cost).

ASLR (Address Space Layout Randomisation). The OS randomises the base addresses of the stack, heap, shared libraries, and mmap region at each execution. This makes it much harder for an attacker to predict where the return address or injected code will be. You can check ASLR status on Linux:

```
$ cat /proc/sys/kernel/randomize_va_space
2    # 0=off, 1=partial, 2=full
```

Non-executable stack (NX/XD bit). The stack is marked as non-executable at the page table level. Even if an attacker manages to inject code there, the CPU will raise a page fault when attempting to execute it.

For an HPC programmer, stack smashing is not primarily a security concern (your codes typically do not process untrusted input). But the underlying mechanism — uncontrolled writes past buffer boundaries — is absolutely relevant because it leads to **silent data corruption**. If you overwrite the return address, you get a crash (which is actually the *lucky* outcome — at least you know something is wrong). If you overwrite a neighbouring local variable by just a few bytes, you get wrong results that may not be detected until the paper is published.

This is why tools like AddressSanitizer (`gcc -fsanitize=address`) and Valgrind (`valgrind --tool=memcheck`) are indispensable in scientific code development. They detect buffer overflows, use-after-free, and other memory errors at runtime with reasonable overhead. Use them routinely during development — the cost of a sanitiser run is negligible compared to the cost of a retracted paper.

Part 4 — The Heap

What the heap is for

The heap is the memory region used for **dynamic allocation**: any data whose size is not known at compile time, or that must persist beyond the scope of the function that creates it, or that is simply too large for the stack.

In C, you interact with the heap through `malloc()` , `calloc()` , `realloc()` , and `free()` . In C++, through `new` / `delete` (or, better yet, through standard containers and smart pointers that manage heap memory for you).

Unlike the stack, the heap has no fixed structure. It is managed by a memory allocator — the default on glibc-based Linux systems is `ptmalloc2` , which maintains multiple "arenas" to reduce contention in multi-threaded programs. For HPC workloads, alternative allocators like `jemalloc` (used by Facebook and Rust's default), `tcmalloc` (Google), or `mimalloc` (Microsoft) can offer better scalability. The choice of allocator can matter: in a massively threaded application, contention on `malloc()` / `free()` can become a bottleneck.

Allocation and deallocation on the heap can happen in any order, which means the heap can become **fragmented** over time — with small free blocks interspersed among allocated blocks. This is one of the performance costs of dynamic allocation, though modern allocators work hard to minimise it.

The heap is where all your large data structures should live: simulation grids, particle arrays, matrices, trees, hash maps. In a well-designed HPC code, the stack holds only lightweight scaffolding (counters, pointers, small temporaries), while the heap holds the actual scientific data.

Stack vs. heap: a comparison

Aspect	Stack	Heap
Allocation	Automatic (compiler-managed)	Manual (`malloc`/`free`) or managed
Speed of allocation	Essentially free (just decrement RSP)	Non-trivial (allocator must find a free block)

Deallocation	Automatic on function return	Manual (`free`), or via destructors/smart pointers
Size	Small (8 MiB default, per thread)	Limited only by available RAM + swap
Scope	Local to the function	Global (accessible from anywhere with a valid pointer)
Fragmentation	None (strict LIFO discipline)	Possible over time
Thread safety	Each thread has its own stack	Shared; allocator must handle concurrency
Cache behaviour	Excellent (small, reused, temporal locality)	Depends on access patterns and data size

Is the stack faster than the heap?

This is a question that comes up often, and the answer is: *it depends, and mostly on how you write the code*. The common misconception is that stack access is inherently faster. Let us look at why this is not exactly true.

When you access an array on the stack, the compiler generates something like:

```
; loop body for on_stack[i] = 0
MOV   eax, DWORD PTR -24[rbp]          ; load loop counter i from stack
CDQE                                     ; sign-extend eax to rax
MOVSS DWORD PTR -1024[rbp+rax*4], xmm0 ; store 0 to on_stack[i]
ADD   DWORD PTR -24[rbp], 1            ; increment i (in memory!)
```

Note: the loop counter `i` is being loaded from and stored back to the stack on every iteration. This is because we compiled without optimisation (`-O0`), and the compiler is faithfully implementing the C abstract machine where every variable lives at its declared location.

For a heap-allocated array accessed through `array[i]` :

```
; loop body for array[i] = 0, where array is a heap pointer
MOV   eax, DWORD PTR -148[rbp]    ; load counter i
CDQE
LEA   rdx, 0[0+rax*4]             ; compute byte offset of element i
MOV   rax, QWORD PTR -36[rbp]     ; load array base pointer from stack
ADD   rax, rdx                     ; compute address of array[i]
MOVSS DWORD PTR [rax], xmm0       ; store 0
```

This has more instructions because we first need to load the base pointer of the array from the stack (it is a local variable holding a pointer to heap memory), then compute the element address by adding the offset.

But if we restructure the loop to use pointer arithmetic directly:

```
float *ptr = array;
float *stop = array + N;
for (; ptr < stop; ptr++)
    *ptr = 0;
```

The compiler can keep the pointer in a register and generate:

```
MOVSS DWORD PTR [rbx], xmm0     ; store 0 to *ptr
ADD   rbx, 4                     ; advance pointer by sizeof(float)
```

This is **fewer instructions** than the stack version. The pointer is already in a register; there is no index computation, no repeated loads of a counter from memory.

The takeaway: the performance difference between stack and heap access is not about some magical property of the stack memory itself — it is about *how* the compiler generates the addressing code. On the stack, variables are addressed relative to a register (RBP or RSP), which is efficient. On the heap, you need a pointer indirection, which adds instructions. But if you manage your pointers well, the heap access can be equally fast or even faster.

With compiler optimisation enabled (`-O2` or `-O3`), the difference essentially vanishes. The compiler promotes heap pointers and loop counters into registers, unrolls loops, vectorises where possible, and generates essentially identical code for both cases. The code for the "optimised pointer" version is approximately what the compiler produces from the naive `array[i]` version when `-O2` is on.

The real performance difference comes from **cache behaviour**: if your working set fits in L1 cache (as a small stack array might), it will be fast. If it does not (as a large heap array likely will not), it will be slower — but that is a property of the *data size and access pattern*, not of where the memory was allocated.

A note on the `calloc()` overhead. In the timing comparison from the slides, the heap version calls `calloc()` which zeroes the allocated memory, while the stack version has no such initialisation cost. The subsequent loop also writes zeros, so the `calloc()` zeroing is redundant work. This is a subtlety related to **memory aliasing** — the compiler may not be able to prove that the `calloc()` and the loop are redundant and may not optimise away the double-zeroing. We will revisit this when we discuss aliasing and the `restrict` keyword in a later lecture.

Part 5 — Practical Exercises and Things to Try

The following exercises help build intuition. They are based on the code examples distributed with the course (in the `examples_on_stack_and_heap/` directory).

Exercise 1: Exceeding the stack (`stacklimit.c`). Check your current stack limit with `ulimit -s`. Compile and run a program that attempts to allocate the full stack size. Observe the segmentation fault — this is the stack guard page doing its job. Slightly increase the stack limit with `ulimit -s` and try again.

Exercise 2: Adjusting the stack programmatically (`stacklimit_setlimit.c`). Use `getrlimit()` and `setrlimit()` to query and enlarge the stack limit from within your code. Observe that the segfault disappears. Then reflect: *should you actually do this in production?* Almost certainly not — it is treating the symptom, not the disease. If you need that much memory, use the heap.

Exercise 3: Comparing stack and heap access times (`stack_use.c`). Measure the time to initialise an array in three scenarios: on the stack, on the heap with `array[i]` notation, and on the heap with pointer arithmetic. Compile with different optimisation levels:

```
$ gcc -O0 -o stack_use stack_use.c && ./stack_use    # no optimisation
$ gcc -O2 -o stack_use stack_use.c && ./stack_use    # with optimisation
```

Without optimisation, the stack version is typically fastest. With `-O2` or `-O3`, the differences shrink or reverse — the compiler generates essentially the same code for all three. This is an important lesson: naive microbenchmarks at `-O0` can give deeply misleading results.

Exercise 4: Returning a pointer to a local variable. The code `stack_use.c` contains a function that deliberately returns the address of a local variable. The compiler warns you. Call this function, then call another function, and inspect the pointed-to data. Understand *why* the data might sometimes appear intact and why relying on that is catastrophically wrong.

Exercise 5: Stack smashing (`stacksmash_simple.c`). Write a simple buffer overflow. Compile with and without stack protection:

```
$ gcc -fno-stack-protector -o smash stacksmash_simple.c
$ gcc -fstack-protector-all -o smash_protected stacksmash_simple.c
```

Observe the difference. Then try with AddressSanitizer:

```
$ gcc -fsanitize=address -o smash_asan stacksmash_simple.c
```

The sanitiser will give you a detailed report of exactly where the overflow occurred, what was overwritten, and where the offending write originated.

Exercise 6: Inspecting the assembly. Use `gcc -S -fverbose-asm` to generate assembly listings for the stack and heap access functions:

```
$ gcc -g -S -fverbose-asm -o stack_use.s stack_use.c
$ gcc -g -S -fverbose-asm -O2 -o stack_use_O2.s stack_use.c
```

Compare the generated instructions and count the operations per loop iteration. For a more informative view that interleaves C source with assembly:

```
$ gcc -g -fverbose-asm -Wa,-adhln stack_use.c > stack_use_detailed.s
```

Getting comfortable with reading assembly is a skill that will serve you throughout this course. You do not need to memorise opcodes, but you should be able to identify loops, memory accesses, register usage, and function calls in compiler output.

Part 6 — Putting It Together: Why This Matters for HPC

At this point you might ask: why spend a lecture on operating system internals in an HPC course? The answer is that everything we will discuss in the rest of the course — cache optimisation, SIMD vectorisation, OpenMP threading, MPI communication — ultimately comes down to how data is laid out in memory and how the hardware accesses it.

Consider these connections:

Memory locality and caches. The distinction between stack and heap, and the choice of data structures on the heap, determines whether your data exhibits spatial and temporal locality. A small, reused buffer on the stack has excellent locality. A scattered linked list on the heap has terrible locality. When we discuss cache-aware programming, the memory model from this lecture is the foundation.

Thread stacks and OpenMP. Every OpenMP thread has its own stack. The default thread stack size (`OMP_STACKSIZE`) is often 2–4 MiB. If your parallel region uses large local arrays, you will get stack overflows — not in the master thread (which uses the process stack), but in the worker threads. This is one of the most common "it

works in serial, crashes in parallel" bugs. Understanding the stack is essential for diagnosing it. *We'll see the details when discussing OpenMP.*

NUMA and memory placement. On NUMA systems (which all modern multi-socket HPC nodes are), the physical location of memory matters. The heap is where your large data lives, and controlling *where* in physical memory it is allocated (through `numactl`, `mbind()`, or first-touch policy) is a critical performance concern. We will return to this when we discuss NUMA-aware programming. *We'll see the details when discussing OpenMP.*

TLB pressure. Scattered heap allocations that touch many pages generate TLB misses. Using huge pages, allocating large contiguous arrays, and traversing them linearly all reduce TLB pressure. The paging model from this lecture explains why.

False sharing. When two threads write to different variables that happen to reside on the same cache line, the coherence protocol bounces the line between cores, destroying performance. Understanding the memory layout — where variables are placed on the stack vs. the heap, and how the compiler pads structures — is essential for diagnosing and avoiding false sharing.

Summary

The main points to take away from this lecture:

Every process on Linux runs in its own virtual address space, which provides isolation and a uniform view of memory. The translation from virtual to physical addresses is handled by the paging mechanism — a multi-level page table hierarchy walked by dedicated hardware in the MMU, with the TLB serving as a critical cache. TLB misses are expensive; huge pages reduce their frequency.

The process memory is divided into well-defined segments: text (code), data (initialised globals), BSS (uninitialised globals, zeroed at startup), heap (dynamic allocation, grows upward), a memory-mapped region (shared libraries, large allocations), and stack (local variables and function call management, grows downward).

The stack is fast, automatic, and limited in size. It is designed for small, transient data. Abusing it with large allocations is a design error that leads to crashes, undefined behaviour, or subtle data corruption. Legacy codes that allocate large arrays on the stack are a recurring source of trouble, especially when parallelised with OpenMP (each thread has its own, typically small, stack).

The heap is where your scientific data belongs. It is flexible, large, and — when used correctly — not inherently slower than the stack. The performance difference between stack and heap access at the instruction level is a matter of addressing modes, not of the memory itself. With compiler optimisation enabled, the difference is negligible.

Accessing memory below the stack pointer is undefined behaviour, modulo the 128-byte red zone defined by the System V AMD64 ABI. Stack smashing — writing past buffer boundaries on the stack — can corrupt return addresses and local data. Modern compilers provide protections (canaries, ASLR, NX bit), but for scientific codes the real concern is not security but silent data corruption. Use sanitisers (`-fsanitize=address`) during development as a matter of course.

Understanding where your data lives and how the hardware accesses it is the foundation for everything that follows in this course.

These notes are based on the lecture slides for HPC 1 at Università di Trieste (2025/2026), and on the lecture's transcriptions.

For code examples, refer to the `examples_on_stack_and_heap/` directory in the course repository. For cultural background on stack smashing, see Aleph One, "Smashing the Stack for Fun and Profit," Phrack 49 (1996), available at https://phrack.org/issues/49/14.